

Interview Question:

A PySpark job in Azure Databricks suddenly takes 3x longer after the data volume

My Answer:

I'd start in the Spark UI rather than changing settings blindly. I'd compare the slow run with a healthy run, then check stage duration, input size, shuffle read/write, task skew, spill, and the physical plan. If one key is creating very large partitions, I'd address skew; if a join is unnecessarily shuffling a small dimension, I'd consider a broadcast join. I'd rerun the same workload and validate both runtime and resource usage.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark transformation is creating millions of tiny output files in ADLS.

My Answer:

I'd first confirm the file-count and partition pattern at the write stage. The usual cause is writing with too many partitions relative to the data volume. I'd tune the number of output partitions before the write, or use coalesce when I only need to reduce partitions without a full shuffle. I'd then check the resulting file sizes and downstream read performance rather than optimizing only for file count.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A join between a huge fact DataFrame and a small reference DataFrame is slow.

My Answer:

I'd inspect the physical plan to see whether Spark is performing a large shuffle join. If the reference dataset is genuinely small enough to fit safely in executor memory, I'd test a broadcast join. I'd also project only the required columns before the join and make sure the join keys have compatible data types. After the change, I'd compare shuffle volume and stage time in Spark UI.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline has severe data skew on customer_id.

My Answer:

I'd identify skew by looking at partition sizes and the frequency distribution of the join key. If a small number of customers dominate the data, a normal hash partition can put too much work on a few tasks. Depending on the use case, I'd isolate heavy keys, use salting for the affected join, or use an appropriate skew-aware strategy. I'd validate that the final result remains correct, not just faster.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job runs out of executor memory during a groupBy.

My Answer:

I'd check whether the aggregation is creating very large in-memory state or whether a skewed key is concentrating records. I'd also inspect whether unnecessary columns are being carried through the pipeline. Rather than immediately increasing executor memory, I'd reduce the data width, filter earlier, and redesign the aggregation if needed. Then I'd verify spill, GC time, task distribution, and final output to confirm the root cause.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A nested JSON column is expensive to process in PySpark.

My Answer:

I'd avoid repeatedly parsing the same JSON string in multiple transformations. I'd define the expected schema and parse it once, then select only the nested fields required by the downstream logic. If the source schema is controlled, I'd also consider storing the structured result in a columnar format such as Delta. I'd compare the physical plan and runtime before and after to make sure the change actually reduced work.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline is using Python UDFs heavily and has poor performance.

My Answer:

I'd identify which UDFs are on the hot path and check whether equivalent built-in Spark SQL functions can replace them. Built-in expressions are generally easier for Spark to optimize than arbitrary Python code. I'd also check whether the UDF is processing more rows or columns than necessary. I'd replace one expensive UDF at a time, benchmark the stage, and keep the change only if correctness and performance both improve.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job creates the same intermediate DataFrame multiple times.

My Answer:

I'd look for repeated actions or repeated transformations that recompute identical data. If the intermediate result is expensive and reused, I'd evaluate caching or persistence, but only after confirming it will fit the workload. I'd also check whether the pipeline can be rearranged so the expensive transformation runs once. I'd validate cache hit behavior and memory pressure because caching everything can make the job worse.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A source system sends duplicate records every day.

My Answer:

I'd first define what makes a record unique, because deduplication without a business key can remove legitimate records. In PySpark I'd typically use the business key and an ordering column such as event timestamp or ingestion timestamp, then keep the desired record with a window or equivalent logic. I'd also measure duplicate rates and validate the result against source totals so the fix is observable.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job fails because a source column changes type.

My Answer:

I'd inspect the actual source values and determine whether the change is legitimate schema evolution or bad data. I'd avoid silently coercing everything to string because that can hide quality problems. Where appropriate, I'd parse into the expected type with controlled handling for invalid values, quarantine bad records, and add a data-quality metric. The important part is making the schema behavior explicit and observable.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A daily PySpark job needs to process only changed records.

My Answer:

I'd first confirm whether the source provides a reliable watermark such as `updated_at`, `version`, or `change sequence`. Then I'd read only the incremental range and maintain a checkpoint or high-water mark outside the transformation logic. For late-arriving changes, I'd define a replay window rather than assuming timestamps are perfectly ordered. I'd compare source change counts with target merge counts on each run.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job reads a large CSV dataset and is slow.

My Answer:

I'd treat CSV parsing as part of the bottleneck and check whether the data can be converted once into a columnar format such as Parquet or Delta. I'd also provide an explicit schema instead of relying on expensive inference when possible. Before writing anything, I'd filter and project only required columns. I'd benchmark parsing time and downstream query time after the conversion.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A transformation uses orderBy on a massive DataFrame and causes a huge shuffle.

My Answer:

I'd confirm whether the global ordering is actually required. A global orderBy forces a distributed sort, which can be expensive at scale. If the requirement is only deterministic ordering within groups, I'd use a partition-aware window or other narrower operation. If global ordering is truly required, I'd keep it but reduce the data before sorting and validate the shuffle volume.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline has many chained withColumn calls and becomes difficult to maintain.

My Answer:

I'd separate the logic into named transformation stages with clear business meaning rather than optimizing syntax alone. I'd also remove columns that are no longer needed and avoid recalculating the same expression. For performance, I'd inspect the optimized plan rather than assuming every withColumn creates a physical pass. The goal is readable code that still produces a sensible execution plan.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark aggregation gives incorrect totals after joining two datasets.

My Answer:

I'd suspect row multiplication before changing the aggregation. I'd compare row counts before and after the join and check whether the join keys are unique on the supposed one side. I'd inspect duplicate keys with a groupBy count and then decide whether to deduplicate, aggregate before joining, or correct the join condition. I'd add a reconciliation test using known totals.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A job has one extremely slow task while the other tasks finish quickly.

My Answer:

I'd treat that as a possible skew or uneven partition-size problem. In Spark UI I'd compare task input records, input bytes, shuffle read, and duration. If one key is responsible, I'd use a skew-handling strategy such as salting or separating heavy keys. If the source partitions are uneven, I'd revisit how the data is partitioned. I'd verify task-time distribution after the change.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline must handle nulls in a financial amount column.

My Answer:

I'd first establish the business rule because replacing null with zero is not automatically correct. If null means missing, I'd preserve it and expose a quality metric; if the business explicitly defines null as zero, I'd apply that rule before aggregation. I'd also use explicit casts and validation for invalid values. Finally, I'd reconcile totals against the source and document the null-handling rule.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A team wants to use `collect()` on a large `DataFrame` for debugging.

My Answer:

I'd avoid `collect` on production-sized data because it brings the full result to the driver and can exhaust driver memory. For debugging, I'd use `limit`, `sample`, `show`, or write a small diagnostic dataset. If I need aggregate diagnostics, I'd compute counts or distributions in Spark and return only the summary. That gives the team visibility without moving the entire dataset to the driver.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job is slow because a filter is applied very late.

My Answer:

I'd check whether the filter can be safely pushed closer to the source and before expensive joins or aggregations. Reducing rows early usually reduces shuffle and memory pressure. I'd confirm the filter is semantically safe and then inspect the physical plan to see whether Spark is already applying predicate pushdown. The validation would compare input records, shuffle volume, runtime, and result counts.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline needs a rolling seven-day metric.

My Answer:

I'd define the event-time semantics first, especially whether late events should change prior results. Then I'd use a windowing approach that matches the required grain and ordering, rather than collecting records to the driver. For streaming data I'd also account for watermarks and state growth. I'd validate boundary dates with a small known dataset because off-by-one time-window errors are common.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline has inconsistent results between runs.

My Answer:

I'd first check for nondeterministic logic such as unordered row selection, random sampling without a fixed seed, or aggregations that assume a stable order. I'd also inspect whether duplicate source files or late-arriving data are changing the input. I'd make record selection deterministic with explicit ordering and stable keys, then compare input snapshots and output checksums across repeated runs.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A DataFrame contains hundreds of columns but only ten are needed.

My Answer:

I'd project the required columns as early as practical. Carrying unused columns through joins, shuffles, serialization, and writes increases I/O and memory pressure. I'd also check whether nested fields can be selected individually instead of materializing an entire complex structure. After the change, I'd compare stage input size and runtime while confirming that no downstream dependency was accidentally removed.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job uses repartition everywhere.

My Answer:

I'd review each repartition because it can introduce a full shuffle. I'd keep repartition where I need a deliberate distribution, such as preparing for a large join or controlling output parallelism, but remove unnecessary calls. If I only need to reduce partitions before writing, coalesce may be more appropriate. I'd use Spark UI and the physical plan to verify whether each shuffle is actually justified.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline needs to write partitioned data to ADLS.

My Answer:

I'd choose partition columns based on common filtering patterns and sensible cardinality, not simply on every available column. I'd avoid very high-cardinality partition keys because they can create too many small directories and files. After the first production loads, I'd inspect file counts, partition sizes, and query patterns and adjust the design if the access pattern changes.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A source has late-arriving events and daily aggregates are already written.

My Answer:

I'd define a correction strategy instead of treating the original daily load as immutable. If the target is Delta, I can recalculate affected dates or use a merge pattern depending on the model. I'd maintain an event-time window that can be replayed safely and make the write idempotent. I'd validate corrected totals for both the late date and unaffected dates.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline must handle malformed records without failing the entire job.

My Answer:

I'd separate valid and invalid records at the parsing boundary. The valid stream would continue through the normal transformation, while malformed records would be captured with enough metadata to diagnose the issue, such as source file and ingestion timestamp. I'd track an error rate and set an operational threshold. If the threshold is exceeded, the pipeline can fail or alert instead of silently accepting bad data.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A large join is slow even after selecting only needed columns.

My Answer:

I'd inspect whether the join keys are causing skew, whether the datasets are much larger than expected, and whether the join type is correct. I'd review the physical plan for exchange stages and check partition sizes. Depending on the evidence, I might use a broadcast join for a truly small side, rebalance a skewed key, or pre-aggregate one side. I'd validate using actual Spark UI metrics.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job spends most of its time in serialization.

My Answer:

I'd look for Python UDFs, large Python objects, unnecessary conversions between JVM and Python, or overly wide rows. I'd replace Python-side processing with Spark-native functions where possible and reduce the data passed between stages. I'd then compare executor CPU, serialization time, and stage duration. I would avoid tuning memory blindly until I understand which object or operation is driving serialization.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A team wants to use pandas inside a distributed PySpark transformation.

My Answer:

I'd clarify the data size and whether pandas is being used on the driver or through a supported distributed pattern. Calling `toPandas` on a large `DataFrame` is risky because it collects data to the driver. If the transformation can be expressed with Spark functions, I'd prefer that. If pandas-style logic is genuinely required, I'd evaluate pandas UDFs or another distributed approach and test memory behavior.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline needs an audit column showing when a row was ingested.

My Answer:

I'd add ingestion metadata at the boundary where the record enters the pipeline, using a consistent timestamp and, where useful, source file or batch identifiers. I'd keep business event time separate from ingestion time because they answer different questions. That metadata helps with replay, debugging, late-arrival analysis, and lineage. I'd validate that retries do not create misleading audit timestamps.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A Spark job is fast in development but slow in production.

My Answer:

I'd compare the environments systematically: data volume, file sizes, partition counts, cluster/runtime configuration, source distribution, and query plan. Development often hides skew and small-file effects because the dataset is too small. I'd reproduce the production-scale pattern with representative data and use Spark UI metrics to isolate the difference. I'd avoid copying production settings into development without evidence.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline needs deterministic deduplication.

My Answer:

I'd define the natural key and the exact tie-breaker, such as latest source timestamp plus ingestion sequence. Then I'd use a window ordered by those fields and keep one record per key. The important part is avoiding an arbitrary first or last record because distributed execution does not guarantee an implicit order. I'd add tests for duplicate keys and tied timestamps.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A job repeatedly scans the same large source for multiple outputs.

My Answer:

I'd first see whether the outputs can share a common filtered and projected DataFrame. If the shared result is expensive and reused, I'd evaluate persistence or a staged Delta result. I'd also consider whether the outputs can be produced in one pass without creating unnecessary materialization. The decision would be based on actual recomputation cost and memory pressure.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline needs to compare current and previous snapshots.

My Answer:

I'd establish a stable business key and compare only the columns relevant to change detection. For large datasets, I'd avoid a blind full cross-comparison and instead join on the business key, then calculate change flags. If the source supports a reliable change sequence, I'd use it to reduce the comparison scope. I'd validate inserts, updates, and unchanged counts against expected totals.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline is failing because of executor lost errors.

My Answer:

I'd inspect the executor logs and Spark UI before changing cluster size. Executor loss can come from memory pressure, container limits, long GC, node issues, or expensive tasks. I'd correlate the failure with a specific stage and task pattern. If the workload is genuinely memory-heavy, I'd reduce data per task or tune the workload first, then scale resources if the evidence supports it.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A DataFrame has a timestamp stored as a string in multiple formats.

My Answer:

I'd profile the formats before parsing so the conversion rule is explicit. I'd normalize the source into a canonical timestamp type and capture values that fail parsing rather than silently converting them to null. I'd also define the timezone assumption because timestamp errors often come from timezone ambiguity. I'd validate a sample of known source timestamps and boundary cases such as midnight and daylight-saving transitions.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A team wants to repartition by a very high-cardinality column.

My Answer:

I'd question the reason first. High-cardinality repartitioning can create many partitions and overhead without improving the workload. If the column is only needed for a join, I'd look at the join strategy and actual distribution instead. I'd use Spark UI to evaluate partition sizes and task counts, then choose a partitioning strategy that balances parallelism with manageable task overhead.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job needs to process a very large number of small input files.

My Answer:

I'd measure file count, average file size, and file-discovery overhead separately from transformation time. If possible, I'd compact the upstream data into healthier file sizes and use an incremental ingestion pattern rather than repeatedly scanning the whole directory. For Auto Loader workloads, I'd also use its supported discovery and checkpointing mechanisms. The goal is to reduce metadata overhead without losing incremental processing semantics.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline needs to validate referential integrity between fact and dimension data.

My Answer:

I'd calculate the number and percentage of fact keys that do not resolve to the dimension. I'd keep the validation separate from the main transformation so the pipeline can report the issue clearly. Depending on the business rule, unmatched records can be quarantined, rejected, or retained with an exception flag. I'd trend the metric over time because a sudden increase usually indicates an upstream change.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark aggregation is correct but unexpectedly expensive.

My Answer:

I'd inspect the number of groups, key cardinality, shuffle size, and whether the aggregation carries wide columns. Sometimes the logic is correct but creates a huge intermediate state. I'd reduce columns before the aggregation, filter earlier, and pre-aggregate where that is semantically safe. Then I'd compare shuffle and task metrics rather than judging the optimization from wall-clock time alone.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A pipeline needs to process only the last N days while preserving older data.

My Answer:

I'd make the date boundary explicit and use the event or business date that actually defines the requirement. I'd filter before expensive transformations and write only the affected target partitions or keys. If updates can arrive late, I'd include a controlled replay window. I'd validate that records outside the window remain unchanged and that the target count reconciles with the source slice.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job has a long stage with many tiny tasks.

My Answer:

I'd check whether the input has been over-partitioned, possibly because of small files or repeated repartitioning. Too many tiny tasks create scheduling overhead and can make a job slower even though individual tasks are cheap. I'd compare task duration and partition count, then reduce unnecessary partitions or compact the source. I'd verify the new task distribution rather than choosing an arbitrary partition number.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A production pipeline must be safely rerun after a partial failure.

My Answer:

I'd design the write path to be idempotent. That usually means identifying the business key or batch identifier and ensuring a retry does not duplicate already-processed records. For Delta targets, I'd use an appropriate merge or overwrite strategy for the affected scope. I'd also persist enough run metadata to know which input slice was processed. Finally, I'd test an intentional failure and rerun the same batch.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A business user reports that a PySpark report changed after a rerun.

My Answer:

I'd compare the input snapshot, code version, configuration, and target state between runs. I'd specifically check for late-arriving records, nondeterministic deduplication, changed reference data, or an incorrect overwrite scope. Once I identify the cause, I'd make the transformation deterministic and add a reconciliation test. I'd explain the result using concrete counts rather than assuming the latest run is correct.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job has excessive garbage-collection time.

My Answer:

I'd inspect executor GC time, heap usage, task data size, and object-heavy transformations. I'd first reduce unnecessary columns, avoid collecting data to the driver, and review Python-side processing. If the workload is still memory intensive, I'd tune partition sizing or executor resources based on the evidence. I'd validate that GC time falls without creating excessive task overhead.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline must handle duplicate source files.

My Answer:

I'd make the ingestion process idempotent by identifying files or batches with stable source metadata and keeping processing state. I'd avoid relying only on row-level deduplication because duplicate files can contain legitimate repeated records. I'd test a replay of the same file and verify that the target does not double-count its records.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark job has an expensive window function.

My Answer:

I'd inspect the window partition and ordering columns because large partitions and global ordering can create substantial shuffle and sort work. I'd reduce the input rows and columns before the window and confirm whether the business logic really needs the full ordering. I'd compare shuffle size, task duration, and output correctness after the change.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline needs a reusable data-quality framework.

My Answer:

I'd define checks such as null thresholds, uniqueness, valid ranges, referential integrity, and row-count reconciliation as reusable functions or configuration-driven rules. Each run would emit pass/fail results with a batch ID. Critical failures could stop publication while noncritical issues are quarantined or alerted. I'd keep the framework separate from business transformations so it remains maintainable.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline must support schema validation across environments.

My Answer:

I'd store the expected schema as a versioned contract and compare incoming data against it before the main transformation. Additive fields can be handled according to the agreed evolution policy, while incompatible type changes should be surfaced explicitly. I'd test the contract in development and production-like data before deployment so environment differences do not become runtime surprises.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark pipeline needs a safe rollback after a bad transformation.

My Answer:

I'd make the output versioned or isolated by batch so a bad run does not overwrite the only valid copy. Before deployment I'd capture the expected row counts and key business metrics. If a bad run occurs, I'd stop downstream publication, restore or replace the affected scope using the supported table mechanism, and rerun after fixing the transformation.

Follow-up Question:

What Spark UI or data-quality evidence would you use to prove that change fixed the issue?

Follow-up Answer:

I'd compare the before-and-after stage metrics, input and output counts, shuffle or task distribution, and a known-good result. I'd keep the change only if the performance improvement also preserves correctness.

Interview Question:

A PySpark join unexpectedly multiplies rows.

My Answer:

I'd stop the downstream aggregation and inspect join cardinality. I'd count duplicate keys on both sides and verify the intended relationship. If one side should be unique, I'd deduplicate or pre-aggregate it using a documented rule, then reconcile row counts and business totals.

Follow-up Question:

What proves the fix?

Follow-up Answer:

Before/after row counts, duplicate-key counts, and a known business total should reconcile.

Interview Question:

A PySpark pipeline receives late events.

My Answer:

I'd separate event time from ingestion time and define an accepted lateness window. For batch, I'd replay the affected event-time range with an idempotent write. For streaming, I'd use event-time and watermark semantics only when they match the business requirement.

Follow-up Question:

What is the common mistake?

Follow-up Answer:

Using ingestion date as business date can place late events in the wrong reporting period.

Interview Question:

A PySpark job is fast in development but slow in production.

My Answer:

I'd compare data volume, file sizes, partition counts, skew, physical plan, and cluster configuration. Small development data often hides shuffle and skew problems. I'd reproduce the production shape before tuning.

Follow-up Question:

Why not just increase the cluster?

Follow-up Answer:

The bottleneck may be data distribution or a bad plan rather than insufficient compute.

Interview Question:

A PySpark job uses toPandas on a large DataFrame.

My Answer:

I'd remove the full collection to the driver. For debugging I'd use sample or limit; for analytics I'd aggregate in Spark and return only the small result. This keeps driver memory bounded as data grows.

Follow-up Question:

What is the risk?

Follow-up Answer:

The entire result is materialized on the driver, which can cause driver-memory failure.

Interview Question:

A PySpark job has one task much slower than the others.

My Answer:

I'd compare task input size, shuffle read, and duration. If one key dominates, I'd investigate skew; if source partitions are uneven, I'd revisit partitioning. I'd fix the distribution rather than blindly adding resources.

Follow-up Question:

What indicates skew?

Follow-up Answer:

A small number of tasks with much larger input or shuffle and much longer duration is a strong signal.

Interview Question:

A PySpark pipeline needs deterministic deduplication.

My Answer:

I'd define the business key and deterministic tie-breaker, such as event time plus ingestion sequence. Then I'd use a window function to retain the intended row. I would not use an arbitrary first or last record.

Follow-up Question:

Why is a tie-breaker important?

Follow-up Answer:

Distributed execution does not guarantee an implicit row order.

Interview Question:

A PySpark pipeline must reduce a very wide DataFrame before an expensive join.

My Answer:

I'd project only the columns required for the join and downstream logic before the expensive stage. Reducing row width can lower shuffle and serialization costs. I'd validate that no required columns are accidentally removed and compare the physical plan and stage metrics before and after the projection.

Follow-up Question:

What evidence would prove the projection helped?

Follow-up Answer:

I'd compare shuffle bytes, stage duration, task input size, executor memory pressure, and total runtime. I'd also validate that the final result is identical.

Interview Question:

A PySpark pipeline needs schema-drift detection.

My Answer:

I'd compare incoming schema with a version-controlled contract and classify additive versus breaking changes. Incompatible changes should block publication and produce an audit record rather than silently changing the target.

Follow-up Question:

What should be logged?

Follow-up Answer:

Source, batch, previous schema, new schema, change type, and pipeline decision.

Interview Question:

A PySpark pipeline must preserve source lineage.

My Answer:

I'd add source file, batch, ingestion timestamp, or equivalent metadata at ingestion and carry it through transformations where traceability is required. This gives operations a direct path from target data back to the source.

Follow-up Question:

Why is lineage useful?

Follow-up Answer:

It speeds reconciliation, incident analysis, and replay.

Interview Question:

A PySpark job has millions of small input files.

My Answer:

I'd separate discovery overhead from transformation time. I'd improve upstream file sizes where possible and use incremental ingestion rather than repeatedly scanning the full directory.

Follow-up Question:

What would you monitor?

Follow-up Answer:

File count, average size, discovery latency, processing rate, and downstream scan behavior.

Interview Question:

A PySpark pipeline needs exact financial calculations.

My Answer:

I'd use an explicit `DecimalType` with documented precision and scale. I'd define rounding at the business boundary and test values near precision limits. Then I'd reconcile totals with the source system.

Follow-up Question:

Why not floating point?

Follow-up Answer:

Floating-point representation can introduce rounding differences that are undesirable for exact financial values.

Interview Question:

A PySpark job has a huge groupBy shuffle.

My Answer:

I'd inspect grouping cardinality, partition sizes, and shuffle read/write. I'd filter and project earlier and remove unnecessary grouping columns. I'd only scale compute after understanding the shuffle bottleneck.

Follow-up Question:

What is the first metric?

Follow-up Answer:

Shuffle volume and partition-size distribution, together with stage duration.

Interview Question:

A PySpark pipeline needs quarantine for invalid records.

My Answer:

I'd split valid and invalid records at the validation boundary. Invalid records should retain source metadata and a reason. I'd monitor the invalid rate and block publication if a critical threshold is exceeded.

Follow-up Question:

Why retain invalid rows?

Follow-up Answer:

They make troubleshooting and reconciliation possible instead of hiding data loss.

Interview Question:

A PySpark pipeline needs source-to-target reconciliation.

My Answer:

I'd capture source count, accepted count, rejected count, target count, and important business measures for each batch. For updates, I'd also reconcile keys because row counts alone can miss duplication.

Follow-up Question:

When should the pipeline stop?

Follow-up Answer:

When a critical reconciliation rule exceeds its agreed threshold.

Interview Question:

A PySpark pipeline unions changing schemas.

My Answer:

I'd align names and data types against a canonical schema before unioning and use `unionByName` where appropriate. I'd explicitly handle missing columns rather than relying on positional order.

Follow-up Question:

Why avoid positional union?

Follow-up Answer:

A source schema change can put values into the wrong target columns without an obvious failure.

Interview Question:

A PySpark left join creates unexpected nulls.

My Answer:

I'd inspect unmatched keys and compare casing, whitespace, and data types. I'd quantify unmatched records before deciding whether null is expected or indicates a source-quality issue.

Follow-up Question:

Would you replace null with UNKNOWN?

Follow-up Answer:

Only if the data model explicitly defines an unknown member; not as a generic fix.

Interview Question:

A PySpark pipeline must calculate a rolling seven-day metric.

My Answer:

I'd define event-time semantics and late-data behavior first. Then I'd use the appropriate window calculation at the required grain and validate boundary timestamps with known data.

Follow-up Question:

What edge case matters?

Follow-up Answer:

Records exactly on the seven-day boundary and late arrivals should be tested.

Interview Question:

A PySpark pipeline needs audit metadata.

My Answer:

I'd store run ID, batch ID, source scope, timestamps, input/output counts, rejected count, and status. I'd keep operational metadata linked to the business data through a stable identifier.

Follow-up Question:

Why is batch ID important?

Follow-up Answer:

It supports reconciliation, replay, and incident investigation.

Interview Question:

A PySpark pipeline needs to replay historical data.

My Answer:

I'd parameterize the exact source range and target scope. The write must be idempotent so a repeated replay cannot duplicate records. I'd test a small slice first and reconcile it.

Follow-up Question:

What makes replay safe?

Follow-up Answer:

Deterministic input scope, idempotent write, and target reconciliation.

Interview Question:

A PySpark job has slow writes but fast transformations.

My Answer:

I'd isolate the sink stage and inspect output partition count, file sizes, concurrent writers, and storage behavior. I'd optimize the write only after proving it is the bottleneck.

Follow-up Question:

What should be compared?

Follow-up Answer:

Write duration, file count, average file size, and representative read performance.

Interview Question:

A PySpark job has an expensive Python UDF.

My Answer:

I'd check whether the logic can be expressed with Spark-native functions. Native expressions generally allow more engine optimization and avoid Python execution overhead. I'd benchmark before and after.

Follow-up Question:

Would you always remove UDFs?

Follow-up Answer:

No. If native functions cannot express the required logic, a UDF may be justified after measuring its cost.

Interview Question:

A PySpark join uses incompatible key types.

My Answer:

I'd normalize the key types explicitly before joining and verify that the conversion preserves business meaning. I'd also check for invalid values created by the cast.

Follow-up Question:

Why not cast inside every join?

Follow-up Answer:

Repeated expressions can add work and hide a schema problem; normalizing at a controlled boundary is easier to govern.

Interview Question:

A PySpark pipeline needs to validate referential integrity.

My Answer:

I'd calculate unmatched foreign keys and report both count and percentage. Depending on the business rule, records can be quarantined, retained with an exception flag, or block publication.

Follow-up Question:

What should be trended?

Follow-up Answer:

The unmatched percentage over time, because a sudden change can indicate an upstream contract problem.

Interview Question:

A PySpark job has repeated actions on the same DataFrame.

My Answer:

I'd check whether the upstream plan is recomputed. If reuse is substantial, I'd evaluate persistence, but only after measuring memory pressure and recomputation cost. I'd remove accidental debug actions from production.

Follow-up Question:

How do you prove persistence helped?

Follow-up Answer:

Compare scans, stage duration, executor memory, and total runtime.

Interview Question:

A PySpark pipeline needs normalized timestamps.

My Answer:

I'd define the canonical timezone and convert at ingestion. I'd preserve source timestamp information where auditability requires it and test daylight-saving and boundary cases.

Follow-up Question:

What is a common mistake?

Follow-up Answer:

Treating a local timestamp as UTC without knowing the source timezone.

Interview Question:

A PySpark pipeline needs top-N per group.

My Answer:

I'd choose ROW_NUMBER when exactly N rows are required and DENSE_RANK when ties should all be returned. I'd include deterministic tie-breakers where necessary.

Follow-up Question:

Why clarify ties?

Follow-up Answer:

The choice of ranking function changes the number of rows returned.

Interview Question:

A PySpark job produces a huge driver result.

My Answer:

I'd look for collect, toPandas, or other driver materialization. I'd replace it with distributed aggregation or a small diagnostic sample.

Follow-up Question:

What is the safe pattern?

Follow-up Answer:

Return only a small summary or sample to the driver.

Interview Question:

A PySpark pipeline must handle empty input.

My Answer:

I'd make an empty batch a valid state when the source contract permits it, while still recording zero input and output counts. If data is mandatory, I'd alert or fail according to the contract.

Follow-up Question:

Why record empty runs?

Follow-up Answer:

It distinguishes a legitimate no-data period from an ingestion failure.

Interview Question:

A PySpark pipeline needs a business-key uniqueness rule.

My Answer:

I'd calculate duplicate counts by business key before publication. If uniqueness is mandatory, I'd block the batch and retain offending records for investigation.

Follow-up Question:

What should the quality report show?

Follow-up Answer:

Duplicate count, affected keys, percentage, batch ID, and source.

Interview Question:

A PySpark pipeline must compare two snapshots.

My Answer:

I'd join current and previous data on the business key and compare only meaningful attributes. I'd classify inserts, updates, deletes, and unchanged records and validate each count.

Follow-up Question:

Why not compare every column?

Follow-up Answer:

Audit fields and ingestion metadata can create false changes.

Interview Question:

A PySpark job needs to process only changed records.

My Answer:

I'd use a reliable source watermark such as `updated_at` or a change sequence and persist the high-water mark. I'd include a controlled replay window for late changes.

Follow-up Question:

What makes the watermark reliable?

Follow-up Answer:

It must be monotonic or used with a replay strategy that prevents missed late changes.

Interview Question:

A PySpark pipeline has malformed JSON.

My Answer:

I'd parse with an explicit schema and capture malformed records with source metadata instead of silently dropping them. I'd monitor the error rate and apply a publication threshold.

Follow-up Question:

Why explicit schema?

Follow-up Answer:

It makes the expected contract clear and avoids uncontrolled inference changes.

Interview Question:

A PySpark pipeline repeatedly repartitions the same DataFrame at several stages.

My Answer:

I'd inspect the physical plan and each repartition boundary to determine whether the partitioning is actually needed. Repartitioning can introduce a full shuffle, so I'd keep only the boundaries that support a downstream join, aggregation, or write requirement. I'd measure the plan rather than removing repartitions blindly.

Follow-up Question:

When is an explicit repartition useful?

Follow-up Answer:

It can be useful before a large distributed operation when it creates a better partition layout for the next stage. The decision should be based on data size, key distribution, and measured shuffle cost.

Interview Question:

A PySpark pipeline needs to compare schemas.

My Answer:

I'd compare names, types, nullability, and nested structures and generate a schema-diff result. Breaking differences should be reviewed before deployment.

Follow-up Question:

Why compare nullability?

Follow-up Answer:

Downstream logic may rely on a field being consistently present.

Interview Question:

A PySpark job has serialization overhead.

My Answer:

I'd look for Python UDFs, large Python objects, driver collections, and JVM/Python conversions. I'd replace Python-side processing with native Spark expressions where possible.

Follow-up Question:

What metric would you inspect?

Follow-up Answer:

Serialization time, executor CPU, stage duration, and the operations crossing the Python boundary.

Interview Question:

A PySpark pipeline needs safe overwrite of one date.

My Answer:

I'd isolate the affected date scope and use a target operation that replaces only that scope. I'd validate source and target counts for the date and confirm older dates remain unchanged.

Follow-up Question:

Why scope the overwrite?

Follow-up Answer:

A broad overwrite can accidentally remove unaffected historical data.

Interview Question:

A PySpark job has inconsistent rerun results.

My Answer:

I'd check nondeterministic row selection, source changes, late data, and ordering assumptions. I'd make record selection deterministic and compare input snapshots across runs.

Follow-up Question:

What should be identical?

Follow-up Answer:

For the same input snapshot and code version, the target result should be deterministic.

Interview Question:

A PySpark pipeline needs a data freshness metric.

My Answer:

I'd compare the latest event or ingestion timestamp with the processing time and define the SLA precisely. I'd publish the freshness value per batch and alert when it crosses the threshold.

Follow-up Question:

Why separate event and ingestion time?

Follow-up Answer:

They describe different operational delays and answer different business questions.

Interview Question:

A PySpark job needs a stable output schema.

My Answer:

I'd select and cast required fields explicitly at the final boundary and avoid exposing accidental upstream columns. I'd validate the schema as part of the deployment test.

Follow-up Question:

Why not `SELECT *`?

Follow-up Answer:

It allows upstream additions or changes to silently alter the contract.

Interview Question:

A PySpark pipeline must support a production rollback.

My Answer:

I'd isolate output by batch or version so a bad run can be identified and replaced safely. I'd record the previous valid version and test the rollback procedure before relying on it.

Follow-up Question:

What is the key design principle?

Follow-up Answer:

Never make the only valid output impossible to recover or reproduce.

Interview Question:

Production PySpark join unexpectedly multiplies rows. — validation case 1

My Answer:

I'd stop the downstream aggregation and inspect join cardinality. I'd count duplicate keys on both sides and verify the intended relationship. If one side should be unique, I'd deduplicate or pre-aggregate it using a documented rule, then reconcile row counts and business totals. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark pipeline receives late events. — validation case 2

My Answer:

I'd separate event time from ingestion time and define an accepted lateness window. For batch, I'd replay the affected event-time range with an idempotent write. For streaming, I'd use event-time and watermark semantics only when they match the business requirement. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark job is fast in development but slow in production. — validation case 3

My Answer:

I'd compare data volume, file sizes, partition counts, skew, physical plan, and cluster configuration. Small development data often hides shuffle and skew problems. I'd reproduce the production shape before tuning. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark job uses toPandas on a large DataFrame. — validation case 4

My Answer:

I'd remove the full collection to the driver. For debugging I'd use sample or limit; for analytics I'd aggregate in Spark and return only the small result. This keeps driver memory bounded as data grows. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark job has one task much slower than the others. — validation case 5

My Answer:

I'd compare task input size, shuffle read, and duration. If one key dominates, I'd investigate skew; if source partitions are uneven, I'd revisit partitioning. I'd fix the distribution rather than blindly adding resources. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

**Production PySpark pipeline needs deterministic deduplication.
— validation case 6**

My Answer:

I'd define the business key and deterministic tie-breaker, such as event time plus ingestion sequence. Then I'd use a window function to retain the intended row. I would not use an arbitrary first or last record. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark job has an expensive window function. — validation case 7

My Answer:

I'd inspect partition and ordering columns because large partitions and sorting can create substantial shuffle. I'd filter and project before the window and confirm the full ordering is necessary. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark pipeline needs schema-drift detection. — validation case 8

My Answer:

I'd compare incoming schema with a version-controlled contract and classify additive versus breaking changes. Incompatible changes should block publication and produce an audit record rather than silently changing the target. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark pipeline must preserve source lineage. — validation case 9

My Answer:

I'd add source file, batch, ingestion timestamp, or equivalent metadata at ingestion and carry it through transformations where traceability is required. This gives operations a direct path from target data back to the source. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.

Interview Question:

Production PySpark job has millions of small input files. — validation case 10

My Answer:

I'd separate discovery overhead from transformation time. I'd improve upstream file sizes where possible and use incremental ingestion rather than repeatedly scanning the full directory. In production, I would also record the before-and-after metric in the batch audit so the improvement is measurable.

Follow-up Question:

What production evidence would you retain?

Follow-up Answer:

I'd retain the run ID, input scope, relevant Spark metrics, output reconciliation, and the final decision so another engineer can reproduce the analysis.